Improving your theming workflow

GUIDE TO COMPONENT-BASED THEMING

CONTENTS

CHAPTER 1

Introduction to Components 5

The Power of Components 5

Component-Based Theming 6

CHAPTER 2

Challenges 10

Getting Buy-in From Your Team 10

Training 10

Upfront Effort 11

Advantages 12

Reusability 12

Improved Team Collaboration 12

Semantic Markup 13

Living Styleguide 13

CHAPTER 3

How to Get Started 14

Updating an Existing Website 15

Recommendations for Existing Projects 16

Starting With a New Website 17

When to Start Discussing Components 18

Working With a Design System 20

Laying the Ground for an Effective Front-End 21

Wiring the Front-End with the Back-End 21

CHAPTER 4

How Can Mediacurrent Help? 23

Documentation and Visual Resources 24

Speaking Engagements 24

Training Workshops 24

Custom Training For Teams 24

Development 25

Consulting 25

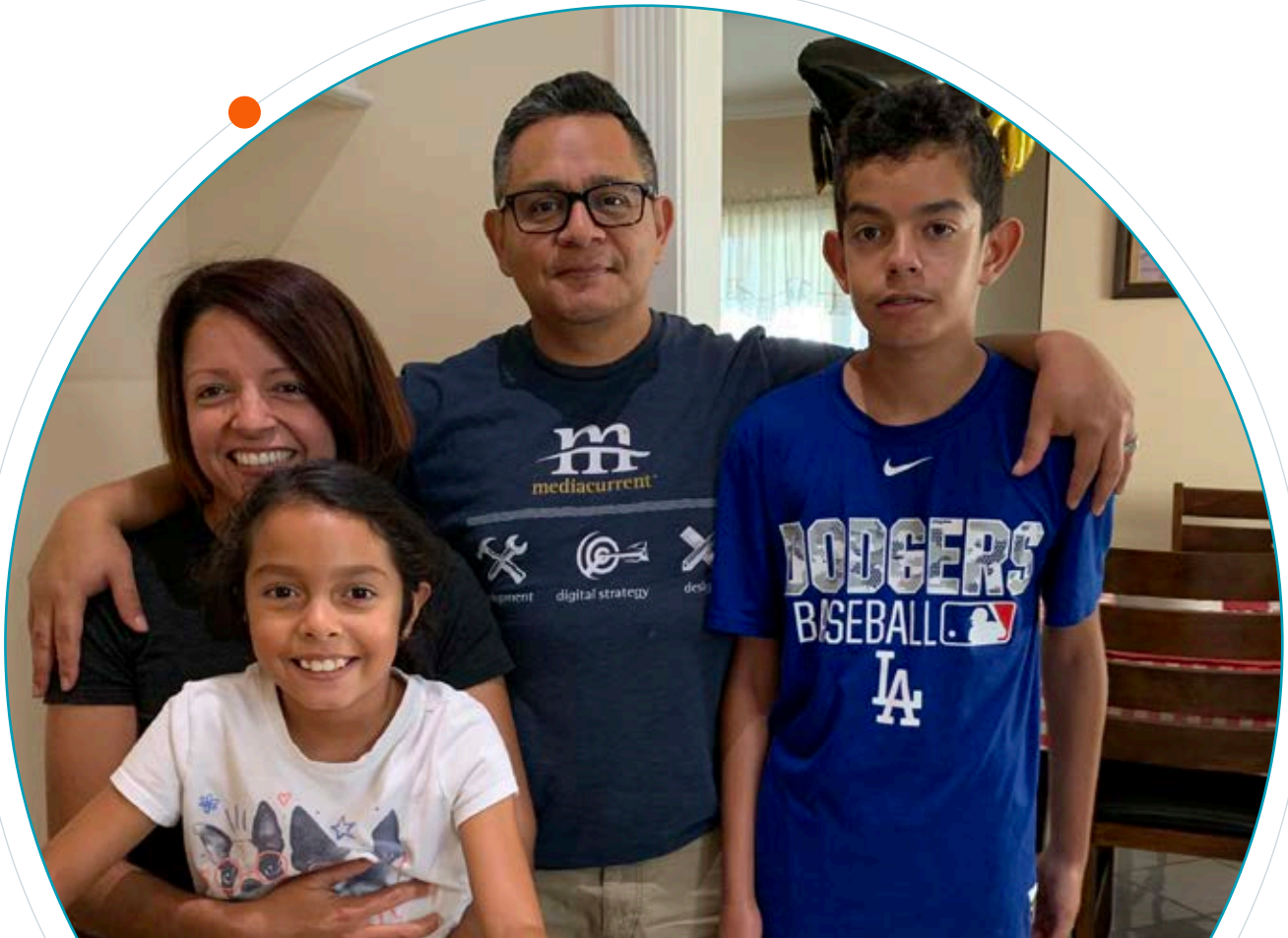
Open Source Tools 26

ABOUT THE AUTHOR

MARIO HERNANDEZ

Mario is the Head of Learning at Mediacurrent with over 10 years of public speaking and training experience. He is a regular speaker and trainer at tech conferences in Los Angeles, San Diego, San Francisco, Washington DC, and others. Mario is a co-host of the Mediacurrent podcast where guest speakers discuss Open Source solutions.

Mario's background as a Front-End Developer has prepared him to mentor and teach others about best practices and standards when building websites. Mario has vast experience in all things Front End as well as Drupal. He developed the very successful Component Based Development training workshop which has been conducted at DrupalCons throughout America and Europe. Mario's strengths are training clients and their development teams through custom-tailored curriculums. Mario lives in Los Angeles with his wife, two kids, and a dog.



CHAPTER 1

Hello and welcome!

Curious about components? You're in the right place. In this guide, you'll find everything you need to get started with developing with components. If you have a hand in your organization's website and digital strategy, this guide is for you.

INTRODUCTION TO COMPONENTS

Think of components as the building blocks of your website. Like a Lego set, one piece can combine with other pieces to form something bigger and more elaborate.

Components can be defined as:

"An elegant, efficient way to organize and deliver content using consistent design patterns that are independent and can be reused."

— Becky Cierpich - UX/UI Designer at Mediacurrent

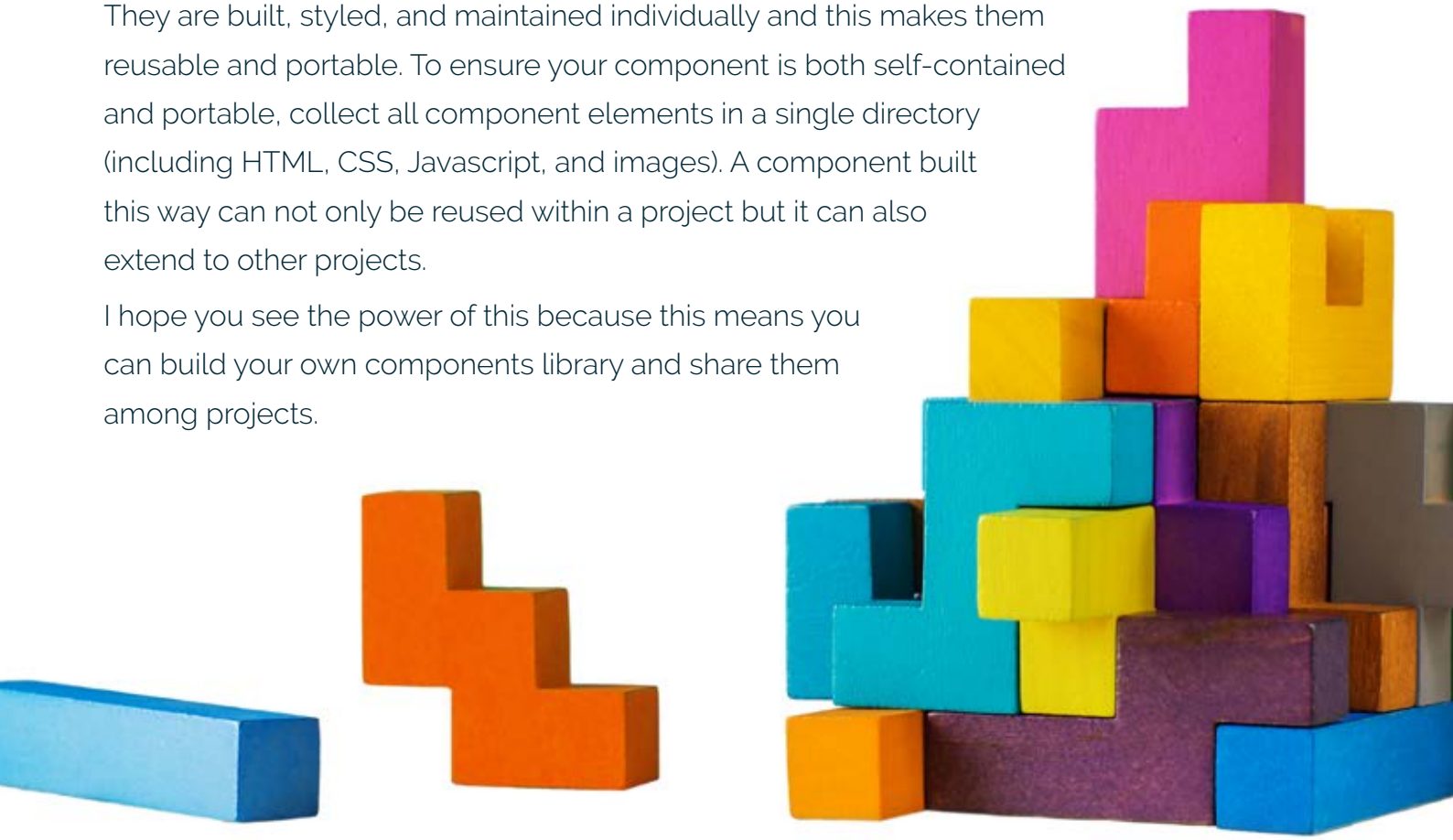
In the context of web development, components open a world of flexibility.

Build your small pieces once and reuse them over and over — anywhere on the site.

THE POWER OF COMPONENTS

The ultimate goal for using components in a web project is to eliminate duplicate code, embrace reusability, and make the code more maintainable long term. It doesn't matter where on the site they are used; all components should look and behave the same way. They are built, styled, and maintained individually and this makes them reusable and portable. To ensure your component is both self-contained and portable, collect all component elements in a single directory (including HTML, CSS, Javascript, and images). A component built this way can not only be reused within a project but it can also extend to other projects.

I hope you see the power of this because this means you can build your own components library and share them among projects.



COMPONENT-BASED THEMING

Now that we've defined components, let's go over the concept of **Component-Based Theming** (or CBT).

> **"A web design or styling approach where you'd select independently built components and combine together according to a predefined system architecture. It relies heavily on a living style guide which serves as the single source of truth for markup, styles, and behaviors of components."**
>
> — Mario Hernandez - Head of Learning at Mediacurrent

At its core, the Component-Based Theming approach means:

1. Breaking your site's interface down into independent pieces.
2. Organizing those pieces into a system of reusable parts. (i.e., the Atomic Design philosophy).
3. Building out those parts in a standalone design system such as Pattern Lab, with a living style guide which becomes the single source of truth for markup, styles, and behavior.

EXAMPLE COMPONENT

Take the example of this simple button. No matter where we use this button, it should look and behave the same. The button component is available for us to use anywhere and as many times as we need without having to create it again.

CLICK ME, I'M A BUTTON

See it in CodePen: http://bit.ly/simple-button

WHAT IF I WANT SOME BUTTONS TO LOOK DIFFERENT?

Most websites use more than one button style. Button styles are oftentimes used to convey meaning to the button's role in a page. The button above, for example, could very well be a website's primary button. It's bold, bright, and stands out. It is normally used as a Call To Action (CTA) button on a component or a page. Let's say we want to have a different style button for more subtle actions like a "Read More" link or a form submission. To do this, we can use a modifier CSS class to update our primary button. We'll call this new button variation "Ghost Button," Ghost because its background is transparent and does not stand out as the primary button.

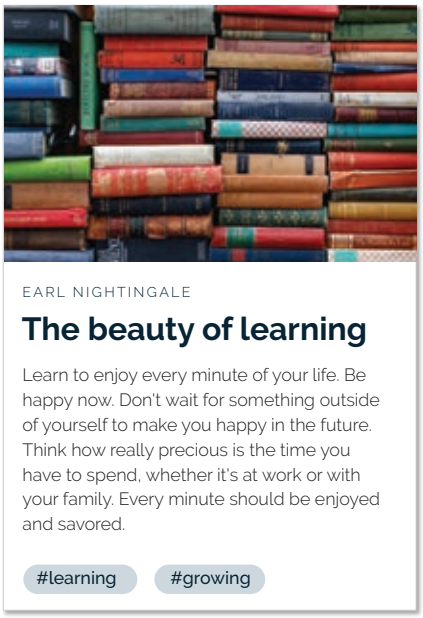
BOO! I'M A GHOST BUTTON

See it in CodePen: http://bit.ly/button-variation

The Ghost button inherits all the styles from the main/primary button, but we've added a new CSS class in addition to button, that class is button--ghost. With this additional class, we can change anything on the ghost button using CSS and Javascript (if needed). Note that we are not rebuilding the button or its styles, we are simply making modifications when both classes button & button--ghost are present on the same element.

A MORE COMPLEX EXAMPLE

The button example may not seem like a huge revelation of Component-Based Theming but it does introduce what's possible with this approach. Let's take a look at a more complex example.



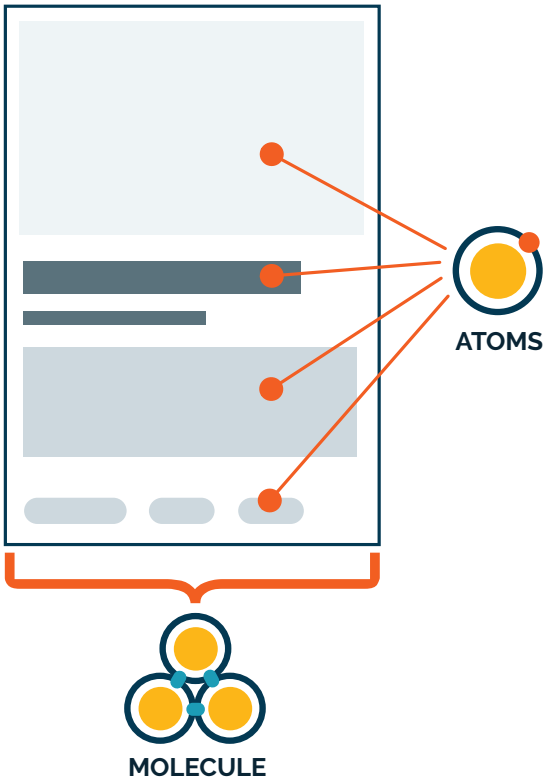
See it in CodePen: https://bit.ly/2MeYJ8I

A component you can almost guarantee on any project is the "Card" component. The reason for this is that the Card is a common way of displaying content in a collected yet visually appealing format. Take the above example: we have a card with a beautiful image, a title, date, teaser text, and some tags. A card like this can be used to display News articles, Events, Media, and more. Within the Atomic Design Methodology, the card would be considered a Molecule because it is typically built by using several Atoms (title, date, image, etc.).

As previously stated, components are self-contained and the Card is a good example of how we can place it anywhere in our site to display all sorts of content. In a News Landing page, it can be used to display a listing of news articles, and on the homepage, it can be used to display a featured news article or event.

Naming components is critical and not always easy because while we want to give components a descriptive name, we want to be aware that we don't paint ourselves into a corner by being too specific with the name. For example, if we were to name the card above "Homepage News Card," we would be limiting ourselves by only being able to use it to display news-type content which lives on the homepage. Technically we could probably use it for displaying non-news content that is not on the homepage, but its name would probably create confusion.

In the next chapter, we will dive deeper into some of the challenges and advantages of components.



CHAPTER 2

Component-Based Theming is slowly becoming the de facto approach for building websites because the traditional top-to-bottom approach is no longer sustainable nor scalable. The old method resulted in duplicate code, poor performance, and lack of semantics — CBT improves on all of these things.

If Component-Based Theming is so great, why doesn't everyone use it?

CHALLENGES

With most new methodologies, we face new challenges. Developing with components, being the paradigm shift that it is, is no exception. Remember when Responsive Web Design was first introduced back in 2010? It was a revolutionary idea to imagine building a single website that would adapt to any device size. Everyone was excited and couldn't wait to start implementing it but it turned out to be a slow process. A lot had to happen for all the pieces to be in place for responsive design to work as promised. The web development community jumped into all kinds of ideas and potential solutions to challenges but there were things not quite ready for it. For example, managing images or media was one of the biggest challenges. Out of that came the ability to use srcset and sizes attributes on images.

Component-Based Theming is no different. As great as it sounds, certain things need to happen before you can begin implementing it on your next project.

You'll likely encounter these three challenges on the path to adopting CBT.

1. GETTING BUY-IN FROM YOUR TEAM

Component-Based Theming is not something you start when you begin coding a project. Get the conversation going as your team is strategizing or going through the discovery process. Getting your team to speak the same language early on results in a more successful implementation and increases the longevity of your project.

It's human nature to resist what we don't understand. Component-Based Theming can be a complicated topic for some stakeholders and members of the web team to grasp. As a result, they may hesitate to adopt CBT as an approach. Educating yourself and your team about this new approach will go a long way to promote acceptance.

2. TRAINING

Where do you go to learn this process? Where do you send your team to learn? With the luxury of time and budget you may be able to take some risks and begin learning by building. You may have team members who can teach others and together become better at it. Sending your team to technology events where such a technique is discussed is a great way to get low-cost training.

Partnering with an agency for customized training and resources is another option.

At Mediacurrent, we know what it's like to start from square one with Component-Based Theming. Just a few years ago, we faced an uptick in client projects wanting to use this new approach. Through hands-on learning, we quickly grasped the basics. With each project, our skills streadily improved. Now, with dozens of CBT projects under our belts, we have the empathy and expertise to teach others about building, styling, and integrating components. Learn more about Mediacurrent's training program in chapter 4.

3. UPFRONT EFFORT

The component-based approach dramatically improves team collaboration, makes your code more reusable and flexible, and boosts the long-term maintainability of your website.

The goal of software development has always been reusability. Build something once and re-use it as many times as needed. So why isn't everyone doing this? In part this is because this approach takes extra time and effort to get started. At first, this can seem counterproductive. This is a real challenge for some because they may not have the time or budget to invest in a project early on. Successful implementation of Component-Based Theming takes time and, in turn, investment before you can see any real deliverables. The pressure to start showing progress right away is sometimes used as an excuse for not adopting such an approach.

Someone who doesn't see the value of this methodology may claim it takes too much time and money to get any tangible progress going. Think beyond short-sighted goals. This methodology proves more successful in the long run. Your investment today will pay off months and even years later when you are doing less development as a result of this approach. More on this later.

ADVANTAGES

We can't ignore the challenges outlined in the previous section and before you can begin reaping the benefits of Component-Based Theming they need to be addressed. So what are the benefits you can expect from implementing this approach?

REUSABILITY

From a technical point of view, the power to building something once and reuse it any time is a game-changer. This is what every developer inspires to do as they write code. When done properly, a component can go a long way and can save you time and effort.

IMPROVED TEAM COLLABORATION

In the past, the typical workflow as a front-end developer was to wait until back-end developers had architected and built Drupal's infrastructure and back-end. This approach blocked front-end developers from getting started right away. Too often, we ended up with complicated, non-semantic code to theme from. By this time we didn't always have the luxury of going back and changing how things were built in the back-end so we had to do the best we could with what we had. This translated into an overly complicated theming process.

With Component-Based Theming as front-end developers, we can build components right out of the gate (even ahead of back-end development) and determine the best possible markup for any given component as well as follow standards, best practices, responsive behavior, etc. This means back-end developers no longer need to guess the markup they need to generate, they can grab the markup directly from the component to build the back-end. As one of our back-end developers said: "It is great to build something in the back-end and have it look and behave as it was intended, this was not the case before Component-Based Theming."

SEMANTIC MARKUP

There is nothing more frustrating for a front-end developer than having to work with bad or non-semantic markup. This is not a case of personal preference. Bad markup can jeopardize the success of a project. When dealing with bad markup (excessive markup with no semantic classes or structure), it feels like you are building a castle in the sand. It feels fragile and with every little touch, something may break. When the markup is not optimal, you can expect the following problems:

- The user experience suffers greatly.
- Choppy and unpleasant responsive design behavior.
- Compromised site performance.
- A tangled path to Accessibility and SEO.
- Repetitive code, or worse, code that is hard to maintain.

LIVING STYLEGUIDE

Whether you are a project manager, developer, or even the client stakeholder, you have a front row seat to a progress indicator for your project: the living style guide.

We can see many benefits to developers and PMs, but let me focus your attention on the client's point of view.

See a transparent view into progress - The client can look to the living style guide to see the team's progression as they work on components.

Take a proactive approach to QA testing - The style guide not only provides a visual representation of the work being done, but it is also a great tool for identifying potential issues and experimenting with a component's functionality and behavior. It can even be used as a way to conduct Quality Assurance (QA) testing.

Prepare for launch - As a client, you no longer need to wait for the entire project to be built to see how something is going to look or behave. You can proactively look at the components and identify issues or potential opportunities for improving it. This is liberating and something our clients love.

See full-page layouts in action - In addition to individual representation of components, a style guide allows us to build full-page prototypes so we can simulate what a given page, like the homepage, will look and how it will behave in different devices. How cool is that?

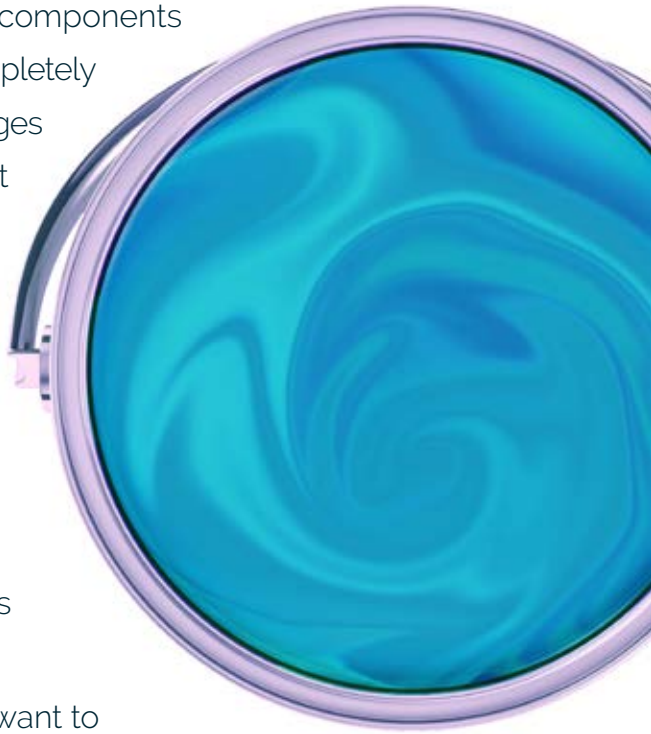
CHAPTER 3

How do you get started?

The concept of Component-Based Theming can be overwhelming as it involves everyone in your team and introduces them to a whole new way of thinking, planning, developing, creating content, and maintaining a high-fidelity system as the project evolves.

The approach you take to get started depends on whether you are building a new website or updating an existing one. Building a new website is the best and recommended way to get started. This clean slate provides the most flexibility for implementing a new methodology. If you are updating an existing website it's more challenging but it is doable.

UPDATING AN EXISTING WEBSITE



It is important to understand that building and styling the components is only half the job. You can do this anytime and completely independent of your current system. The technical challenges you will experience are related to adapting your current system to the new components. This is the part where major changes will need to be made. Before you can make use of the new components your system's back-end needs to be refactored to comply with the new component approach. Using Drupal as an example of a back-end system, you may have content types in place which will need refactoring to become individual components. This can be done by detaching certain fields into separate groups.

Let's say you want to build the Card component we demoed in chapter 1, you could build either a new content type, or paragraph type, or a custom block called Card. This new entity would then have the same fields of the Card component, and finally, it would be integrated by creating a Twig template suggestion (also known as Presenter Template). in which you will pass the Drupal data to the component.

Also, if your website architecture changes, which there is a high possibility that it will, you need to account for internally migrating your content into the new infrastructure.

This is a simple example of the impact you can expect if you decide to implement Components on your existing website. There are so many other considerations to make based on the uniqueness of your system.

RECOMMENDATIONS FOR EXISTING PROJECTS:

TAKE INVENTORY OF YOUR WEBSITE

Maybe not every part of your website needs to be touched with new components. Perhaps you can identify pages or pieces of functionality that are good candidates for components and the rest can be left alone. Navigation, Footer, Header are good candidates if they are not too complicated to refactor. Content listing or list items can also be helpful as they would cover more ground by repeating the same pattern across different types of content.

THINK REUSABILITY FROM THE GET-GO

Before you start building components, take a close look at your existing website. See if you can identify patterns that will allow you to build fewer components and reuse them on multiple areas. You may need to put together some wireframes or design mockups to properly plan out the components you will build and where they can be used. This could be a good opportunity to improve functionality, accessibility, and UX.

START SMALL

If you want to dip your toes in the water start with simple things to get a feel for what you're getting yourself into. Social icons, footer or other small parts of your website may be good candidates to get you started before approaching the bigger things in your site. Or maybe there's something new you've been wanting to do for a while and haven't been able to do until now. This may be good as it won't affect current functionality, but instead will add new functionality.

FORMS AND TABLES DON'T PLAY WELL WITH COMPONENTS

Don't waste your time building components for forms or tables. Using global styles for both can get you very far in ensuring your form elements and tables have a unique look and feel.

IT'S ALL OR NOTHING

Let me explain. If you have decided Component-Based Theming is the way to go, the parts you do convert to components need to be fully maintained using the living style guide, even if you don't convert your website 100% into a component-based website. On a project where components were not the original approach, it may be tempting to continue to maintain your website the old way and slowly neglecting the style guide you worked so hard to build. Once you start adopting components, your style guide should be the single source of truth for markup, styles, and behavior of your components. Don't give in to the temptation to style or modify things outside of your style guide.

DECOUPLED FRONT-END

Consider a decoupled front-end approach. With this setup, your back-end doesn't have to change as much. Instead, your components' data is manipulated through JSON or another format outside of Drupal. This approach presents its own set of challenges that are outside the scope of this guide but may be a way to avoid a complete rewrite of your back-end infrastructure.

NEW WEBSITE

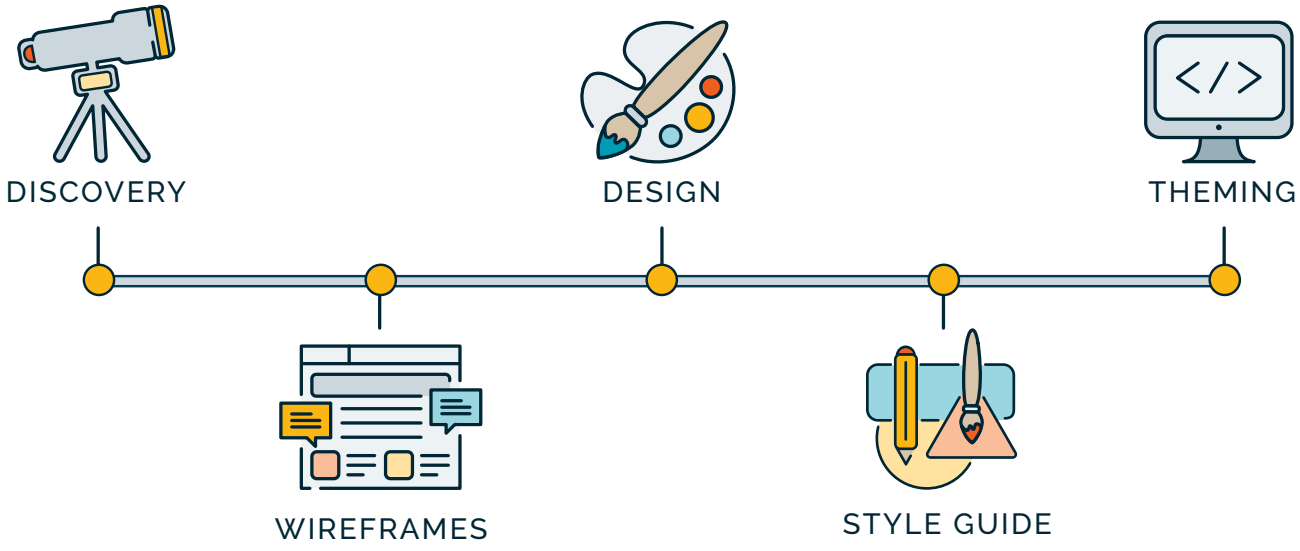
Starting a new project using the component-based approach is very exciting. It is an opportunity to build something great that can make the life easier for the builders, users, and everyone involved. But as the saying goes, *"with great power comes great responsibility"* because you may be tempted to start building components right away before any preparationt work has been done, but jumping ahead coud jeopardize your project.

Remember, this new approach is not your typical approach. You will get better results if you can have discussions about components with your team early on and don't rush into development.

If this is the first time building a new website with components, there are a lot of things to be done that you normally would not do on a traditional project. Not only will you be introduced to new tools and processes, but your team will also need to learn about components and what it means for each of their respective disciplines (i.e. Strategists, Project Managers, Content Managers, Designers, Developers, QA/Testers, etc.). You can expect that shaking up the status quo will take some adjustment.

WHEN TO START DISCUSSING COMPONENTS

This is a question we get asked frequently and it's not surprising because at first is not clear the perfect time to begin the discussions about components. There is a misconception that components discussions don't start until it is time to begin coding. This is not only not recommended but it could also be dangerous because waiting until you are ready to code would be the equivalent of waiting until you start building the walls of a new house to decide how many rooms, windows, doors, etc. it should have. The chart below is a good visual for when components should be discussed with the different stakeholders in your team.



DISCOVERY/CONTENT STRATEGY

The Discovery or Content Strategy phase is the ideal time to kick off these conversations. As your team maps out a content straetgy, they need to be aware that components may present new opportunities to optimize site content for better targeting. With this information at hand, digital strategists can now communicate their goals for the design team to begin putting together some wireframes.

WIREFRAME LIBRARY

Wireframes typically go through an iterative process where new opportunities to improve the experience can be found as well as potential challenges that can be addressed early on before they make it into designs. With a solid set of wireframes, designers can now begin creating high fidelity designs that focus on the visual aspects of the project while paying close attention to content flow, responsive behavior, and User Experience (UX). As you can see throughout this iteration process we are learning and identifying ways in which our product can be even better. Imagine not having these discussions this early in the process, we would miss so many opportunities to improve or catch potential issues.

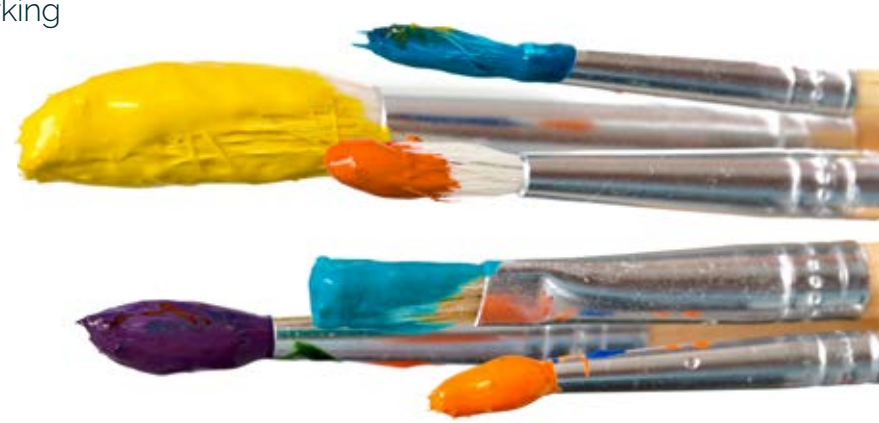
DESIGN/STYLE GUIDE/THEMING

With designs that account for all device sizes as well as accessibility, Front-end developers can be certain that what they are tasked to code has gone through a rigorous acceptance process and there is very little room for something not working as expected. Front-end developers typically work closely with designers to ensure both teams are on the same page.

Designers with strong HTML/CSS knowledge can catch or identify potential issues on wireframes or design comps. The main goal here is to ensure content flows naturally as the content is viewed on mobile or large devices. This may not always be obvious if the designer does not fully understand the natural content flow, semantic markup or how responsive design works. Knowing the expectations for markup as content transitions from mobile to desktop is key. Forcing the content flow on a page as it transitions from mobile to desktop, or vice versa can lead to a less than optimal theming experience as these changes may require moving content around by repeating content and showing/hiding as needed or using CSS positioning techniques to place content into place. This is not a good approach as it affects UX and accessibility among other things.

WORKING WITH A DESIGN SYSTEM

A design system is a front-end framework used to centralized your development workflow. I like to think of design systems as a collection of reusable components, constants, and best practices that help teams to build modern applications with ease. They make it easy to architect the front-end of a project and establish rules for your team to follow when writing code. The flavor of design system you pick depends largely on the programming language you are comfortable with and/or are currently working with. There are many features within a design system but probably the most appealing is the Living Style Guide.



The style guide plays an essential role when building website components. In addition to serving as the primary destination for showcasing a project's guidelines for branding and documentation, the style guide catalogs all components built on the project. Even more importantly, the style guide should be the single source of truth for components' markup, styles, and behaviors. This means that as tempting as it may be to resort to old habits of building and styling pages directly within Drupal (or any other CMS you may be using), you need to instead do this in the style guide to ensure it stays up to date. There is no bigger harm to your component-based project then neglecting the style guide and letting it become obsolete or inconsistent with your project.

LAYING THE GROUND FOR AN EFFECTIVE FRONT-END

When it comes to writing code for a project using the component-based approach, the biggest responsibility falls into the hands of front-end developers. Most front-end code will be authored by FE developers and this will set the tone for the project. This is a huge responsibility that is best handled by a team. Together, the team designated to this task must be well-versed and knowledgeable of the following:

- Writing proper and semantic markup when building components.
- Solid experience with the latest CSS standards and best practices.
- Solid understanding of Responsive Design. Proper markup ensures the responsive behavior is natural and not forced.
- Ability to create flexible and fluid layouts. This goes hand-in-hand with responsive design.
- Ability to manage media to techniques on how to serve the right assets for the job at hand
- Writing optimized Javascript.
- Techniques to improve performance and provide a great user experience.

WIRING THE FRONT-END WITH THE BACK-END

In traditional web development, back-end work needs to be completed before front-end work can start. Front-end developers need an environment to do their work, and without a style guide, the only place is directly in the CMS. In modern web development, and in particular in a component-based theming environment, this issue is solved because now front-end developers can work in the design system's style guide. This means that front-end developers can work side by side and, in some cases, ahead of back-end developers.

The most common approach for integrating the newly built components with Drupal, for example, is to use presenter templates. In the context of component integration, presenter templates are the template suggestions you typically would create

for customizing how content is rendered in Drupal. An example of a presenter template, or template suggestion, would be node--article.html.twig. Presenter templates serve as a bridge between Drupal and the components. Presenter templates' main purpose is to pass Drupal data to the component's twig template. That's pretty much it.

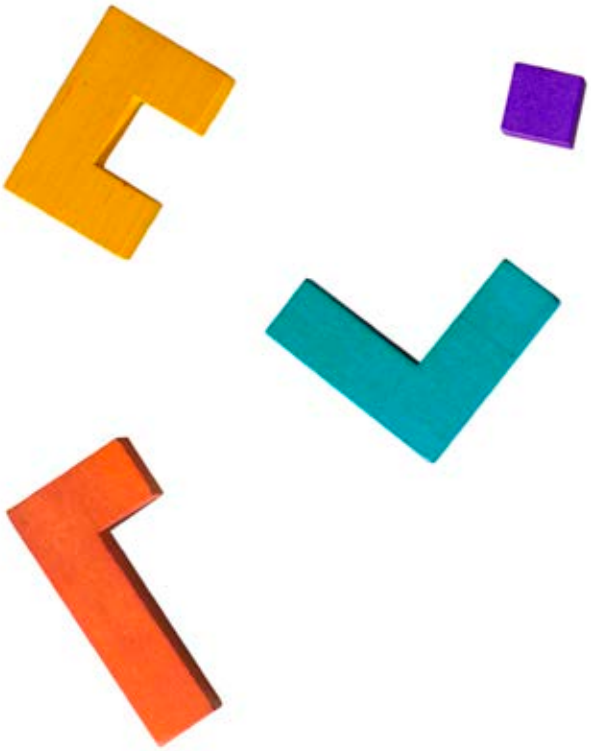
Here's an example of how a presenter template would allow for Drupal's content to be passed to a component.

```
{%
  include '@patterns/card/card.twig' with {
    title: label,
    body: content.body|render|trim is not empty ? content.body
  } only
%}
```

The example above shows how a component named Card is integrated with Drupal by using a presenter template (i.e. block--card.html.twig). First, we type an include twig statement in which we pass the path to the Card component. The **with** keyword allows us to specify the fields of the component we want to make available. In this example, we are mapping the title and body fields and passing Drupal data to them. The **only** keyword allows us to limit the fields so even if the Card component had more fields, we are opting to only pass the title and body fields.

The main takeaway from the example above is that although we are creating a template suggestion for a block, we are only using it to pass data to a component we built in the design system.

We are letting all the work we did with the component drive the look and feel of this specific element on the page (Card). Should the component need updating we would do so in the design system and if needed we would update the template suggestion. This ensures that the design system where all components are built remains the only source of truth for markup, styles, and behavior.



CHAPTER 4

How can Mediacurrent help?

As we've explored in this guide, the new trend of component-based development introduces new challenges, tools, and technologies. At the same time, the advantages of this approach makes it possible to write reusable code, help teams work better together, and create flexible projects that are easier to maintain in the long term.

At this point, you may have some lingering questions. If the complexity of getting started with CBT feels overwhelming, you are not alone. So where do you go from here? This is where Mediacurrent comes in. With years of experience with this approach, we can offer the services you need as well as resources to learn more about such a revolutionary approach for developing websites.

DOCUMENTATION AND VISUAL RESOURCES

We have years worth of content available on our blog (mediacurrent.com/blog), written by numerous authors from different disciplines in the industry and we are certain you will find the content appealing. Also, we have conducted webinars on most topics related to Component-Based Theming which you can access via our video resources page.

SPEAKING ENGAGEMENTS

If you attend a Drupal or tech event anywhere chances are you will see someone from the Mediacurrent team speaking. We are active speakers and love sharing our experiences with the community. This is a great way to get to know our team and to ask them questions directly.

TRAINING WORKSHOPS

Our presence as trainers at Drupal events continues to set the bar for providing high-quality training on everything from components, accessibility, SEO, Gatsby, and more. All of our trainers offer years of hands-on experience. Our training programs reflect the latest standards and practices you would expect from a cutting-edge agency like ours. We encourage you to attend our training workshops at the various camps throughout the states as this is a great and low-cost way to be exposed to the latest trends in development.

CUSTOM TRAINING FOR TEAMS

If you are looking for a training program tailored for your team, we can do that for you. This is not a cookie-cutter training, we analyze your specific needs and write a curriculum unique to your training requirements. Typically we would use real-life tasks or projects to train your team so that they can apply what they learned, while they learn it, directly to the

project with which they're involved. This will ensure you get the most out of the training and cover the topics of interest to you. Our typical custom training varies from 2 - 4 days depending on the level you wish to cover or if you plan on combining more than one discipline (Back-end, Front-End, Strategy, Accessibility, etc.). Having your development team trained by ours will ensure you can begin seeing results right away. Our one-on-one approach has proven successful for clients.

DEVELOPMENT

Our team of experts has years of hands-on experience on everything discussed in this guide and are ready to become an extension to your team. Whether you are looking for a React/Gatsby based decoupled website or a full-blown enterprise-level system, our team has the track record to face any challenge and ensure your project is a success.

If you hire Mediacurrent to build your project but also want your team to be part of the development process, we would train your team prior to starting development so they can begin contributing to the project from day one. This is a great way to ensure your team is leveled up and ready to take ownership of your project when our job is done

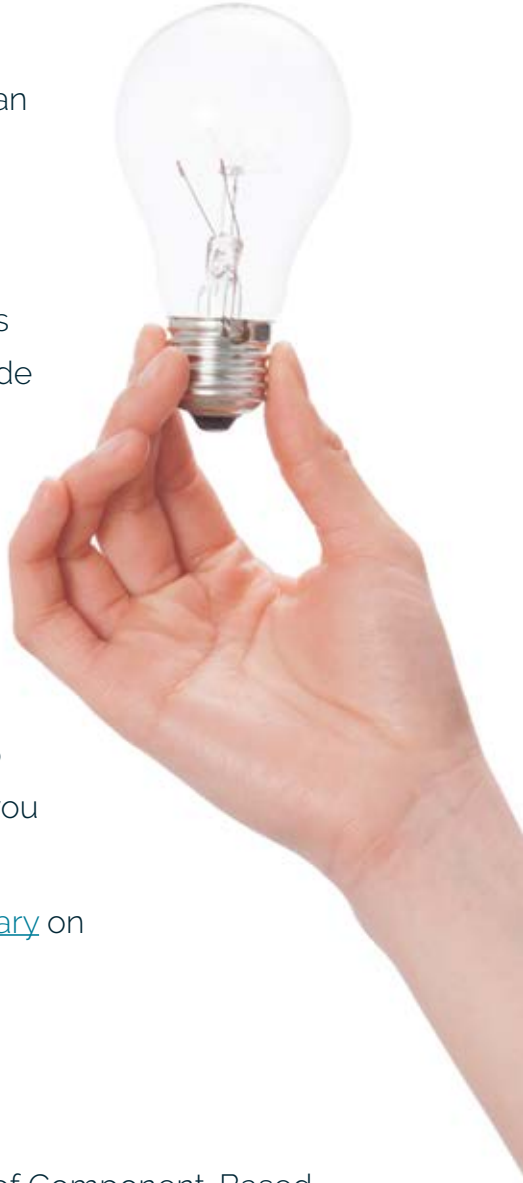
CONSULTING

If you have a team that is trained and feels confident they can do all the work, we can still provide consulting services in the event you run into challenges where additional insight is needed.

OPEN SOURCE TOOLS

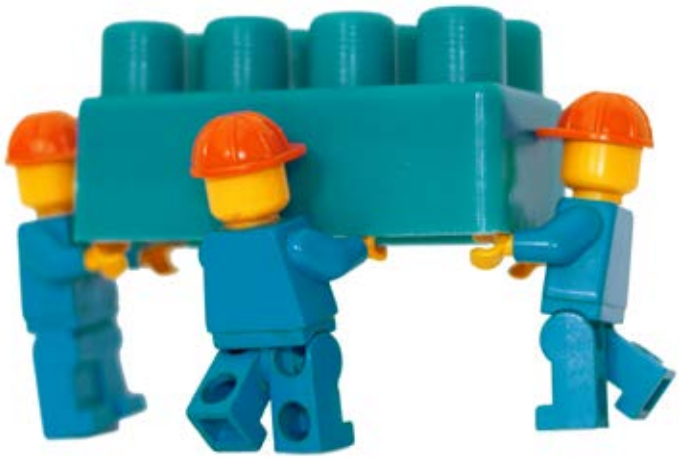
Last but certainly not least, we are always working on tools that can help your organization jump-start your component-based project. Some of the most popular tools include:

1. **Mediacurrent's Theme Generator:** This Front-End tool allows developers to quickly spin off a Drupal 8 theme which includes automated workflows for code linting, media compression, code compilation and more. This is a node-based framework that uses the latest version of Pattern Lab along with Gulp, NPM, and others. The Theme Generator can also be used for non-Drupal projects.
2. **Mediacurrent's Rain Install Profile:** With years of research and testing, Mediacurrent has gathered some of the most powerful and used features of Drupal and packaged them into an open source and enterprise level installation profile which you can begin using now. Get Rain.
3. **Custom Icons Library:** Use of Mediacurrent's custom icon library on your next web or print project.



IN CLOSING

We hope this guide has helped you get a better understanding of Component-Based Theming. Although it can be an overwhelming topic to dive into, the great news is that a lot of smart people are doing it and in the process creating all kinds of resources and tools to make your life easier. This is an evolving approach and it is often changing but having a good understanding of the basics will ensure you are able to adapt to the changes. We hope this guide has armed you with the basics to start experiencing the advantages of this great approach to building websites.



mediacurrent

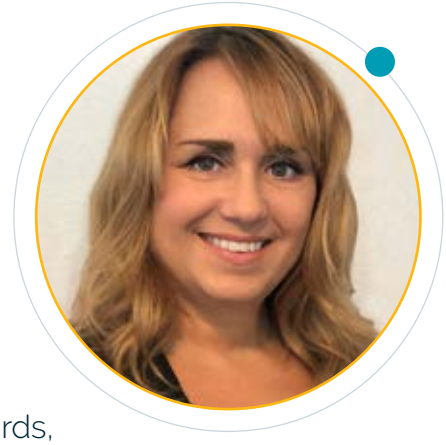
CONTRIBUTORS

TARA ARNOLD

In her role as Senior Marketing Content Strategist, Tara drives the creation of resources to promote Mediacurrent as a thought leader in the technology space. She thrives on the small business imperative to work smart through strategic marketing automation, digital marketing, and creative brand building.

Prior to joining Mediacurrent, Tara spent five years sharpening her inbound marketing skills in the higher education technology and publishing industries. Her specialties include inbound marketing, event marketing, webinars, blogging, and marketing automation. She holds a BA in Communications (Public Relations focus) and minored in creative writing.

CHERYL LITTLE



As the Senior Director of User Experience, Cheryl leads the Mediacurrent Digital Strategy and Design teams by working to develop a solid foundation of processes, services, and body of work. A champion of inclusive design, Cheryl strives to build engaging, targeted, user-centered experiences.

With more than 20 years of experience in design, recent accolades for Cheryl's work include (4) W3 Awards, (2) AIVA Awards, and a UPCEA Award.

MEGAN HARRIS

Megan is a Digital Strategist who thrives on collaboration, clear communication, and transparency for her clients. Her extensive digital marketing background began 7 years ago and has encompassed a wide variety of sectors including SEO, Copy Strategy, Content Marketing, and Social Media.

The work she is most proud of is helping companies gain freedom from worrying about their digital marketing. She says "it's truly been very rewarding to be part of teams over the years who have customer service at the forefront."

ABOUT MEDIACURRENT

Mediacurrent is a full-service digital agency that implements world class open source software development, strategy, and design to achieve defined goals for enterprise organizations seeking a better return on investment.

If you'd like to speak with a representative from Mediacurrent, contact us today.

mediacurrent